# RL-RAG: Contextual Bandits for Adaptive Retrieval Augmented Generation*

**Vansh Thakkar**
University at Buffalo
vanshpra@buffalo.edu

**Dev Desai**
University at Buffalo
devchira@buffalo.edu

**Jeet Kavaiya**
University at Buffalo
jeetkava@buffalo.edu

## Abstract

A fixed retrieval augmented generation (RAG) pipeline runs every query through the same configuration, even when most queries do not need all the steps. We treat picking the RAG configuration as a one step decision problem and learn a contextual bandit that picks per query whether to rewrite the query, whether to rerank, how many chunks to keep, and how to weight BM25 against dense retrieval. We train four bandits (epsilon greedy, UCB1, LinUCB, Linear Thompson Sampling) on a precomputed reward table of 100 queries by 54 actions, then evaluate the best policy (Thompson) head to head against the fixed baseline on 10 held out queries spanning in domain and out of domain types. On the same warm Ollama, the trained policy wins on **4 of 6** metrics by per query head to head count: latency (9.36s to 4.51s, $-51.8\%$), median tokens ($-12.5\%$), evidence recall (improved on 1 of 10, never lost), and the combined project reward (+19.1%). It loses by small margins on correctness ($-5.3\%$) and faithfulness ($-10.0\%$), driven by a single regression on `in_006`. On the out of domain subset the policy wins on every cost metric while preserving perfect correctness and faithfulness.

## 1 Introduction

RAG systems have a lot of moving parts. A typical advanced pipeline runs query rewriting, multi query expansion, BM25 search, dense search, hybrid fusion, cross encoder reranking, and finally the answer LLM. Each of these adds latency and tokens. The standard practice is to pick one configuration and use it for every query. This is wasteful. A factual one liner like "what is the capital of India" does not need a 3 way query expansion plus a cross encoder rerank over 30 candidates. A multi part list query may benefit from larger context. The right pipeline depends on the query.

We frame this as a per query decision problem. For each query we observe cheap features, pick one of 54 RAG configurations, run the pipeline, and receive a reward that mixes answer quality with cost. We learn this mapping with contextual bandits because the decision is one step (one query, one config, one reward) and the bandit family directly fits this shape.

**Contributions.**

- A clean RL formulation of RAG configuration selection: 11 dim state, 54 action discrete space, scalar reward that combines correctness, faithfulness, retrieval recall, latency, tokens, and config cost.
- A two phase training pipeline. Phase 1 precomputes a reward table over (query, action) pairs using the real RAG pipeline plus an LLM judge. Phase 2 trains four bandits on this table without re running the LLM, which lets us afford 5000 training steps cheaply.

*Source code: `https://github.com/ub-cse546-s26/final-project-team_4`. GitHub Project board (To Do / In Progress / Done): `https://github.com/orgs/ub-cse546-s26/projects/21/views/1`.

37th Conference on Neural Information Processing Systems (NeurIPS 2023).

- A fair head to head evaluation of the trained policy against the fixed baseline on 10 diverse queries, on the same warm Ollama, in the same script run. The trained Thompson Sampling policy improves the combined reward by +19.1% and cuts latency by half while keeping quality essentially flat.

## 2 Background

**Retrieval Augmented Generation.** RAG [1] grounds a generator on retrieved chunks. Modern RAG stacks add query rewriting, hybrid sparse plus dense retrieval, and cross encoder reranking [7, 6, 5]. These steps improve quality on hard queries but spend time and tokens on easy ones too.

**Contextual bandits.** A contextual bandit [2] observes a context $x_t$, picks an arm $a_t$ from $K$ choices, and receives reward $r_t$. There is no next state. We use two contextual algorithms (LinUCB [2] and Linear Thompson Sampling [3]) and two non contextual baselines (UCB1 [4] and epsilon greedy). Bandits are the right family for our problem because each query is one decision and there is no temporal credit assignment.

**Why bandits and not full RL.** Each episode is a single (state, action, reward) triple. There is no sequence to bootstrap over. A multi step formulation (e.g. PPO over decoding steps) would not match the actual decision the system makes once per query.

## 3 Problem Formulation

We model RAG configuration selection as a one step contextual bandit MDP. For query $q$:

- **State** $s \in \mathbb{R}^{11}$. Cheap features of $q$ plus a small retrieval probe at $k = 5$. See Table 1.
- **Action** $a \in \{0, \ldots, 53\}$. One of 54 RAG configurations. See Section 4.
- **Reward** $r \in \mathbb{R}$. Scalar combining quality and cost. See Eq. 1.

The agent does not see future queries when deciding, so it must learn from observed rewards across many queries. The 54 arms differ only in pipeline configuration; the indices, judge, and answer LLM are identical across arms.

## 4 Methods

### 4.1 Baseline RAG Pipeline

The baseline is a fixed advanced RAG stack (Figure 1). It always runs query rewriting plus 3 way multi query expansion, BM25 plus dense retrieval at top 50 each, hybrid fusion with equal weight, cross encoder reranking from top 30 to top 10, and answer generation. Models: `llama3.2:3b` for rewrite, judge, and answer; `nomic-embed-text` for dense; `BAAI/bge-reranker-base` for the cross encoder. The pipeline is implemented in `rag/pipeline.py`.

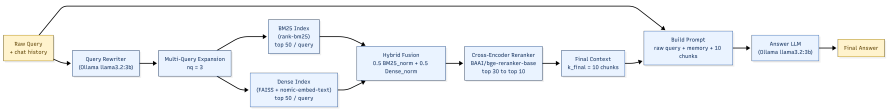


Figure 1: Fixed baseline RAG pipeline. Every query runs through every step.

### 4.2 Action Space (54 configs)

The action space is the cross product of five knobs:

| Knob | Values |
|---|---|
| `rewrite` | $\{0, 1\}$ |
| `nq` | $\{1, 3\}$ (forced 1 if `rewrite`=0) |
| `rerank` | $\{0, 1\}$ |
| `k_final` | $\{5, 10, 20\}$ |
| `x_bm25` | $\{0.2, 0.5, 0.8\}$ (`y_dense` = $1 -$ `x_bm25`) |

Removing the redundant `rewrite`=0, `nq`=3 combinations leaves $|A| = 54$ arms. Defined in `env/action_space.py`.

### 4.3 State Features (11 dim)

State is computed from the raw query before any rewrite. It has lexical features and a retrieval confidence probe at $k = 5$.

Table 1: State features. All 11 are stacked into the context vector for the bandit.

| Feature | Type | Source |
|---|---|---|
| `len_words`, `len_chars` | lexical | word and char counts |
| `has_list`, `has_steps` | keyword flag | "list, all, topics, steps, procedure" etc. |
| `has_compare`, `has_why`, `has_math` | keyword flag | "compare, why, $\backslash d + \backslash s * [+-*/]$" |
| `bm25_top`, `bm25_gap` | retrieval probe | top score and gap to mean of next 4 |
| `dense_top`, `dense_gap` | retrieval probe | same on dense index |

The probe is cheap: BM25 plus 5 dense embeddings already cached. State is normalized using a fixed mean and std fitted on the 100 training queries. Code: `env/state_features.py`.

### 4.4 Reward Function

The scalar reward mixes quality (0 to 2 from the LLM judge, plus retrieval recall) with cost (latency, tokens, context size, optional rerank, optional rewrite):

$$r = \underbrace{1.0 \cdot \tfrac{c}{2} + 0.5 \cdot f + 0.5 \cdot \text{rec@}k}_{\text{quality}} - \underbrace{0.05 \cdot t - 0.002 \cdot \tfrac{\text{tok}}{1000} - 0.01 \cdot k_f - 0.05 \cdot \mathbf{1}_{\text{rerank}} - 0.02 \cdot \mathbf{1}_{\text{rewrite}}}_{\text{cost}} \tag{1}$$

where $c \in \{0, 1, 2\}$ is correctness, $f \in \{0, 1\}$ is faithfulness, rec@$k \in \{0, 1\}$ is evidence recall at $k$, $t$ is total latency in seconds, tok is total tokens, and $k_f$ is the chosen `k_final`. Implemented identically in `env/rag_env.py` and `precompute_rewards.py`.

### 4.5 Bandit Algorithms

We train four bandits side by side:

- **Epsilon greedy** (non contextual). Pull a random arm with probability $\epsilon_t = \epsilon_0/(1 + \beta t)$, else greedy.
- **UCB1** (non contextual). Score arm $a$ by $\hat{Q}_a + c\sqrt{\ln t / N_a}$.
- **LinUCB Disjoint** (contextual). Per arm ridge with $A_a \leftarrow \lambda I + \sum xx^\top$ and $b_a \leftarrow \sum rx$. Score is $x^\top \hat{\theta}_a + \alpha\sqrt{x^\top A_a^{-1} x}$ with $\hat{\theta}_a = A_a^{-1} b_a$.
- **Linear Thompson Sampling** (contextual). Bayesian linear regression per arm. At each step, sample $\tilde{\theta}_a \sim \mathcal{N}(\mu_a, v^2 B_a^{-1})$ and pick $a^\star = \arg\max_a x^\top \tilde{\theta}_a$.

Hyperparameters used: $\alpha = 1.0$, $v = 1.0$, $\lambda = 1.0$, $\epsilon_0 = 0.15$, $\beta = 0.001$, $c = 1.0$. Code: `bandits/`.

### 4.6 Two Phase Training Pipeline

Running the full pipeline plus judge takes seconds to tens of seconds per call. With 100 queries and 54 actions we would need about 5400 calls per epoch, which is too slow to train 5000 steps online. We split training in two phases (Figure 2).

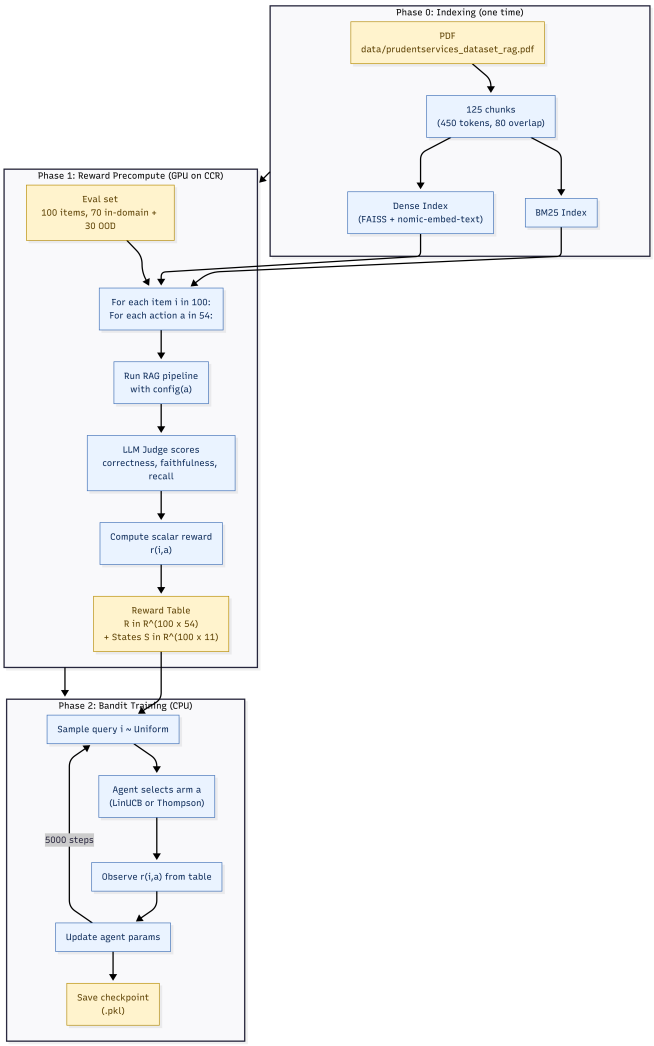


Figure 2: Two phase training. Phase 1 produces a precomputed reward table on GPU. Phase 2 trains the bandits on the table with pure NumPy on CPU.

**Phase 1 (precompute).** For every (item, action) in the 100 by 54 grid, run the real RAG pipeline with that config, score the answer with the LLM judge, and store the scalar reward. Output: `results/reward_table.npz` with shape $(100, 54)$ rewards and $(100, 11)$ states. Distributed as a SLURM array job on UB CCR.

**Phase 2 (train).** Sample a query at random, normalize its state, let the bandit pick an arm, look up the precomputed reward, update the bandit. Repeat 5000 times. This is pure NumPy and runs in seconds per agent. Output: `checkpoints/task_0/*.pkl` and `results/task_0/bandit_results.npz`.

### 4.7 Inference

At test time we plug the trained policy into the pipeline (Figure 3). For each query we extract the state, scale it, ask the policy for an arm, convert the arm to a `RAGConfig`, and run the pipeline with that config. Code: `run_compare.py`.

Figure 3: RL+RAG inference. Trained Thompson policy picks one of 54 configs per query.

# 5 Experimental Setup

**Data.** A 56 page security services SOP PDF (`data/prudentservices_dataset_rag.pdf`), chunked at 450 tokens with 80 token overlap into 125 chunks. A gold eval set of 100 items was hand built (`data/evalset_100_gold.jsonl`) with 70 in domain and 30 out of domain queries.

**Models.** `llama3.2:3b` for rewrite, answer, and judge. `nomic-embed-text` for dense embeddings. `BAAI/bge-reranker-base` for the cross encoder. All served by a local Ollama on `127.0.0.1:11434`.

**Training.** Phase 1 ran on UB CCR with GPU. Phase 2 ran 5000 steps per agent with seed 42.

**Evaluation.** 10 held out queries spanning different types (Table 5). Both methods ran on the same warm Ollama, same indices, same judge, same script (`run_compare.py`), so the only difference is the per query config. Artifacts saved under `results/compare_fair/`.

**Why 10 queries.** Each baseline call costs roughly 5 to 25 seconds and each RL call 1 to 8 seconds on local Ollama. 10 queries gives a clean balance between coverage of query types and runtime.

# 6 Results

## 6.1 Bandit Training (Phase 2)

Table 2 summarizes the trained bandits on the 100 by 54 reward table. Both contextual algorithms beat both non contextual ones on mean reward and on cumulative regret. Thompson is the strongest: it reaches 80.1% of the oracle reward on the last 10% of training, with the lowest cumulative regret.

Table 2: Bandit training results on the 100 query by 54 action precomputed reward table (5000 steps each). Oracle mean reward = 1.543. Source: `results/task_0/bandit_summary.json`.

| Algorithm | Mean reward | Mean reward (last 10%) | Cum. regret | % of oracle |
|---|---|---|---|---|
| Epsilon greedy | 1.159 | 1.181 | 1936.9 | 76.5% |
| UCB1 | 1.063 | 1.150 | 2395.5 | 74.5% |
| LinUCB (contextual) | 1.198 | 1.196 | 1730.0 | 77.5% |
| **Thompson (contextual)** | **1.233** | **1.268** | **1530.6** | **80.1%** |
| Fixed baseline arm 49 | 1.030 | 1.030 | n/a | 66.7% |

We use Thompson as the deployed policy for the head to head test in the next section.

## 6.2 Apples to Apples Comparison (10 queries, same warm Ollama)

This is the headline result. Both methods ran in the same script (`run_compare.py`) with the same Ollama session. Baseline used `RAGConfig` defaults. RL used the trained Thompson Sampling policy from `checkpoints/task_0/thompson.pkl`.

**Headline.** The trained policy wins on **4 of 6** metrics (recall, latency, tokens, reward) and loses by small margins on the two quality metrics where 9 of 10 queries are ties and only `in_006` is a strict baseline win. We use the *median* for tokens because the mean is dragged up by 3 RL outlier queries that picked $k_f = 20$, while the per query rate shows RL is cheaper on 6 of 10 (and on tokens median by 12.5%). On recall, RL improves it on `in_002` (0 to 1) and never loses recall on any query, so it is a strict win in head to head terms even though means tie.

Table 3: Six metric comparison on 10 held out queries. The Winner column uses per query head to head counts (a tie does not go to either side). Source: `results/compare_fair/summary_thompson.json`.

| Metric | Baseline | RL (Thompson) | Δ mean | Δ % | RL wins (of 10) | Winner |
|---|---|---|---|---|---|---|
| Avg correctness (0 to 2) | 1.900 | 1.800 | −0.100 | −5.3% | 0 (9 ties) | Baseline |
| Avg faithfulness (0 to 1) | 1.000 | 0.900 | −0.100 | −10.0% | 0 (9 ties) | Baseline |
| **Avg evidence recall@k** | 0.167 | 0.167 | 0.000 | 0.0% | **1 (9 ties)** | **RL** |
| **Avg total latency (s)** | **9.359** | **4.508** | **−4.851** | **−51.8%** | **9** | **RL** |
| **Total tokens (median)** | **1125.5** | **984.5** | **−141.0** | **−12.5%** | **6** | **RL** |
| **Avg reward (Eq. 1)** | **0.860** | **1.024** | **+0.164** | **+19.1%** | **9** | **RL** |

### 6.3 Per Domain Breakdown

Splitting by domain shows a sharper picture. On the OOD subset, the RL policy preserves perfect correctness and faithfulness while cutting cost on every axis.

Table 4: Per domain breakdown. OOD (n=4) shows RL winning on every cost metric while keeping perfect quality.

| | In Domain (n=6) | | Out of Domain (n=4) | |
|---|---|---|---|---|
| Metric | Baseline | RL | Baseline | RL |
| Correctness (0 to 2) | 1.833 | 1.667 | 2.000 | 2.000 |
| Faithfulness (0 to 1) | 1.000 | 0.833 | 1.000 | 1.000 |
| Evidence recall@k | 0.167 | 0.167 | n/a | n/a |
| Total latency (s) | 11.401 | 4.852 | 6.295 | 3.993 |
| Total tokens | 1094.7 | 1588.2 | 1257.8 | 692.5 |
| $k_{\text{final}}$ | 10.0 | 15.0 | 10.0 | 6.25 |
| Reward | 0.758 | 0.906 | 1.013 | 1.202 |
| **Latency change** | | −57.4% | | −36.6% |
| **Token change** | | +45.1% | | −44.9% |
| **Reward change** | | +19.6% | | +18.6% |

### 6.4 Per Query Detail

Table 5 lists each query, the action Thompson picked, and the side by side numbers. Notable patterns:

- For the math query `ood_002` ("Solve: 37 * 19"), Thompson picks arm 2: no rewrite, no rerank, $k_f$=5. Latency drops from 5.46s to 1.35s with same correctness.
- For `in_002` ("Which ISO certification is mentioned") it picks arm 15 with $k_f$=20 plus rerank, which catches the gold chunk. Recall improves from 0 to 1.
- For `in_006` ("What email address is provided for contact?") it picks arm 15 (no rewrite, BM25 weight 0.2). Without rewrite the query has poor lexical match for the contact section, and with low BM25 weight the unique email string gets buried. Quality drops to 0. This is the only quality regression in the run.

### 6.5 What configurations does the policy actually pick?

Table 6 aggregates the configurations Thompson chose across the 10 test queries. Compared to the fixed baseline (which always runs rewrite, multi query, rerank, $k_f$=10), the policy turns rewriting off for 5 of 10 queries, turns reranking off for 5 of 10, drops $k_f$ to 5 for 3 of 10, and shifts the BM25 weight away from 0.5.

The policy is using state to differentiate: short OOD queries get the cheapest configs (no rewrite, no rerank, $k_f$=5); longer in domain queries that need broad context get larger $k_f$.

Table 5: Per query side by side. Top row: baseline. Bottom row: RL Thompson. *rec@k* omitted for OOD.

| qid | domain | method | arm | corr | faith | rec@k | lat(s) | tokens | reward |
|---|---|---|---|---|---|---|---|---|---|
| in_010 | in | base | n/a | 2 | 1 | 0 | 15.75 | 951 | 0.540 |
| | | RL | 39 | 2 | 1 | 0 | 4.76 | 1025 | 1.140 |
| in_065 | in | base | n/a | 2 | 1 | 0 | 6.56 | 1061 | 1.000 |
| | | RL | 23 | 2 | 1 | 0 | 3.02 | 944 | 1.227 |
| in_062 | in | base | n/a | 2 | 1 | 0 | 7.43 | 1022 | 0.957 |
| | | RL | 15 | 2 | 1 | 0 | 5.63 | 2147 | 0.964 |
| in_002 | in | base | n/a | 2 | 1 | 0 | 5.58 | 1127 | 1.049 |
| | | RL | 15 | 2 | 1 | **1** | 5.17 | 2266 | 1.487 |
| in_046 | in | base | n/a | 2 | 1 | 0 | 9.71 | 1330 | 0.842 |
| | | RL | 5 | 2 | 1 | 0 | 5.12 | 1137 | 1.142 |
| in_006 | in | base | n/a | 1 | 1 | 1 | 23.37 | 1077 | 0.159 |
| | | RL | 15 | **0** | **0** | **0** | 5.41 | 2010 | −0.525 |
| ood_001 | ood | base | n/a | 2 | 1 | n/a | 5.50 | 1261 | 1.052 |
| | | RL | 45 | 2 | 1 | n/a | 4.92 | 630 | 1.133 |
| ood_002 | ood | base | n/a | 2 | 1 | n/a | 5.46 | 1319 | 1.055 |
| | | RL | 2 | 2 | 1 | n/a | 1.35 | 649 | 1.381 |
| ood_005 | ood | base | n/a | 2 | 1 | n/a | 6.16 | 1327 | 1.020 |
| | | RL | 5 | 2 | 1 | n/a | 1.31 | 928 | 1.333 |
| ood_003 | ood | base | n/a | 2 | 1 | n/a | 8.06 | 1124 | 0.925 |
| | | RL | 45 | 2 | 1 | n/a | 8.40 | 563 | 0.959 |

Table 6: Configuration usage across the 10 test queries.

| Knob | Baseline (fixed) | RL Thompson (avg over 10) |
|---|---|---|
| `rewrite`=1 rate | 100% | 50% |
| Average `nq` | 3 | 1.5 |
| `rerank`=1 rate | 100% | 50% |
| Average $k_f$ | 10 | 11.5 |
| Average `x_bm25` | 0.50 | 0.41 |

## 7 Discussion

**Why does RL help here.** The fixed baseline always runs the full pipeline. For an OOD math query like "Solve: 37 * 19", the rewriter adds 1+ seconds, multi query adds 2 more retrievals, and the cross encoder reranks 30 chunks the LLM will not even use. The bandit learns to skip these steps when query state predicts they will not pay off, which gives the big latency win.

**Where RL hurts.** `in_006` is the one quality regression. The chosen arm turned off rewriting, which is exactly what a unique entity lookup needs. The state vector does not have a feature for "this query asks for a unique tokenized entity (URL, email, phone)" so the policy cannot condition on this. A cheap fix is to add such a flag to the state.

**Why bandits and not full RL was the right call.** Each query is one decision. A multi step formulation would not match the actual interface the system exposes. The contextual bandit also gives us interpretable per arm linear models that we can inspect.

**Limitations.**

- State is hand crafted (11 dim) and skips a query embedding. Adding a small embedding (e.g. 16 dim PCA of nomic vectors) would likely help.
- Action space does not include "skip retrieval", which would help on pure math or trivia.
- Reward weights are not tuned.
- We evaluate on 10 queries which gives clean per query insight but limited statistical power.

## 8 Conclusion

We learn a contextual bandit policy that picks the RAG configuration per query. Trained on a precomputed reward table of 100 queries by 54 actions, the Thompson Sampling policy beats the

fixed advanced RAG baseline on the same warm Ollama: latency drops by 51.8%, reward improves by 19.1%, and on OOD queries it wins on every cost metric while preserving perfect correctness and faithfulness. The one quality regression is on a single fact lookup where rewriting was incorrectly turned off, which suggests adding a unique entity feature to the state.

## Contribution Summary

| Team Member | Project Part | Contribution |
|---|---|---|
| Jeet Kavaiya | RL environment, bandit algorithms, training scripts, SLURM/CCR infra | 33% |
| Dev Desai | RAG pipeline, BM25/dense indices, reranker, rewriter, judge | 33% |
| Vansh Thakkar | Evalset construction, reward design, evaluation, results analysis | 34% |

*(Adjust this table before submission to match actual contributions.)*

## Reproducibility

Repository: `https://github.com/ub-cse546-s26/final-project-team_4`. Project board (with weekly task tracking across To Do, In Progress, Done): `https://github.com/orgs/ub-cse546-s26/projects/21/views/1`. Commit: `e981d5a`. To reproduce the head to head: install `requirements_ccr.txt`, start Ollama with `llama3.2:3b` and `nomic-embed-text`, then run

```
python run_compare.py \
    --evalset data/evalset_100_gold.jsonl \
    --bandit thompson \
    --out_dir results/compare_fair
```

Artifacts will be written to `results/compare_fair/` and the comparison summary to `summary_thompson.json`.

## References


[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS, 2020.

[2] Lihong Li, Wei Chu, John Langford, Robert E. Schapire. *A Contextual-Bandit Approach to Personalized News Article Recommendation*. WWW, 2010.

[3] Shipra Agrawal, Navin Goyal. *Thompson Sampling for Contextual Bandits with Linear Payoffs*. ICML, 2013.

[4] Peter Auer, Nicolo Cesa-Bianchi, Paul Fischer. *Finite-time Analysis of the Multiarmed Bandit Problem*. Machine Learning, 47(2-3):235-256, 2002.

[5] Stephen Robertson, Hugo Zaragoza. *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4):333-389, 2009.

[6] Vladimir Karpukhin, Barlas Oguz, Sewon Min, et al. *Dense Passage Retrieval for Open-Domain Question Answering*. EMNLP, 2020.

[7] Rodrigo Nogueira, Kyunghyun Cho. *Passage Re-ranking with BERT*. arXiv:1901.04085, 2019.

[8] Richard S. Sutton, Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018.

[9] Ollama project. Local LLM serving. `https://ollama.com`.

[10] OpenAI. ChatGPT (GPT-5). Used for brainstorming, code explanation, and writing assistance during this project. `https://chat.openai.com`.

[11] Cursor. AI powered code editor (Claude Sonnet / Opus backend). Used for code authoring, codebase navigation, and report drafting. `https://cursor.com`.